INFORME FINAL

Modelado de amenazas de EventosU

Tarea	Modelado de amenazas de un sitio web
Caso	EventosU (sitio ficticio, en fase de diseño)
Integrante 1	Christian Sosa — 22000102 (GitHub: ChristianSosa22)
Integrante 2	Pablo Siquinajay — 2200647 (GitHub: Pablokill2004)
Herramienta	OWASP Threat Dragon 2.x, metodología STRIDE
Modelo	`model/eventosu.json`
Repositorio	https://github.com/ChristianSosa22/eventosu-threat-model
Fecha	Noviembre de 2026

2. Alcance, activos y supuestos

2.1 Alcance

EventosU todavía no tiene código. Todo el análisis se razona sobre la descripción del caso y los supuestos de la sección 2.3; no se ataca ningún sistema real.

Dentro del modelo

Elemento	Qué se analiza
Sitio web (React en CDN)	Manejo del token de sesión en el navegador, entrada de datos del usuario
Panel de administración (`/admin`)	Control de acceso por rol, gestión de usuarios
API (Node.js en contenedor)	Autenticación, autorización, lógica de inscripción, pago, validación de QR y exportación
Base de datos PostgreSQL	Integridad y confidencialidad de usuarios, inscripciones y pagos
Bucket de objetos	Afiches y CSV exportados
App de Puerta (móvil)	Validación de entradas y lo que guarda el dispositivo
Integraciones	Los flujos hacia y desde Microsoft Entra ID, la pasarela de pagos y el servicio de correo
Registros de auditoría	Dónde se guardan y quién puede alterarlos (supuesto S1)

Fuera del modelo

Elemento	Por qué queda fuera
Seguridad interna de Entra ID, la pasarela y el proveedor de correo	Son terceros; solo se modela cómo EventosU confía en lo que le envían
Datos de tarjeta	El caso indica que EventosU no los almacena; los procesa la pasarela (supuesto S6)
Infraestructura del proveedor de nube y del CDN	Responsabilidad del proveedor; sí se modela la configuración que EventosU controla (bucket, red)
Dispositivos personales de los usuarios	Malware en el navegador o el teléfono del estudiante no es controlable por EventosU
Seguridad física del recinto	Saltar una barda o entrar por otra puerta no depende del sitio web
Entradas falsificadas en papel sin QR	No pasan por el sistema

2.2 Activos

ID	Activo	Tipo	Valor para el negocio
A1	Credenciales y sesiones (hashes de contraseña, JWT, clave de firma del JWT)	Dato	Quien las obtiene actúa como cualquier usuario, incluido el administrador. Su pérdida invalida la confianza en todo el sitio.
A2	Datos personales de inscritos (nombre, correo, carné, teléfono)	Dato	La universidad es responsable de ellos. Una fuga expone a estudiantes a phishing y daña la reputación institucional.
A3	Registros de inscripción y pago (estado "pagada", monto, referencia de la pasarela)	Dato	Son la prueba de quién pagó. Si se alteran, la universidad pierde ingresos y no puede resolver reclamos.
A4	Entradas QR	Dato	Equivalen a dinero: cada QR válido es un cupo de un evento. Una entrada duplicada o falsificada deja fuera a quien sí pagó.
A5	Función de validación de ingreso	Función	Si falla el día del evento se forman colas o se deja entrar sin control; es el momento más visible del servicio.
A6	Catálogo de eventos y afiches	Dato	Es la cara pública de la universidad. Un afiche alterado puede difundir información falsa o enlaces maliciosos con la marca institucional.
A7	CSV exportados de inscritos	Dato	Concentran en un solo archivo los datos personales de todo un evento; son el objetivo más rentable para una fuga masiva.
A8	Secretos de integración (secreto del webhook, llave del servicio de correo, client secret de Entra ID, credenciales de la base de datos)	Dato	Permiten suplantar a EventosU ante terceros o a los terceros ante EventosU.
A9	Roles y privilegios (función de gestión de usuarios)	Función	Deciden quién puede crear eventos, exportar datos o administrar. Un cambio indebido de rol abre todos los demás activos.
A10	Registros de auditoría	Dato	Sin ellos no se puede investigar un incidente ni responder a un reclamo de pago o de ingreso.
A11	Cupo de los eventos (inventario de entradas)	Función	Un evento que aparece "agotado" por reservas falsas no vende entradas reales.

2.3 Supuestos

El caso no define estos puntos. Se usan de forma consistente en el DFD, las amenazas y el árbol de ataque.

ID	Supuesto
S1	La API escribe sus registros (accesos, cambios de rol, validaciones de QR, exportaciones) en un servicio de logs administrado de la misma nube. Se modela como un almacén separado.

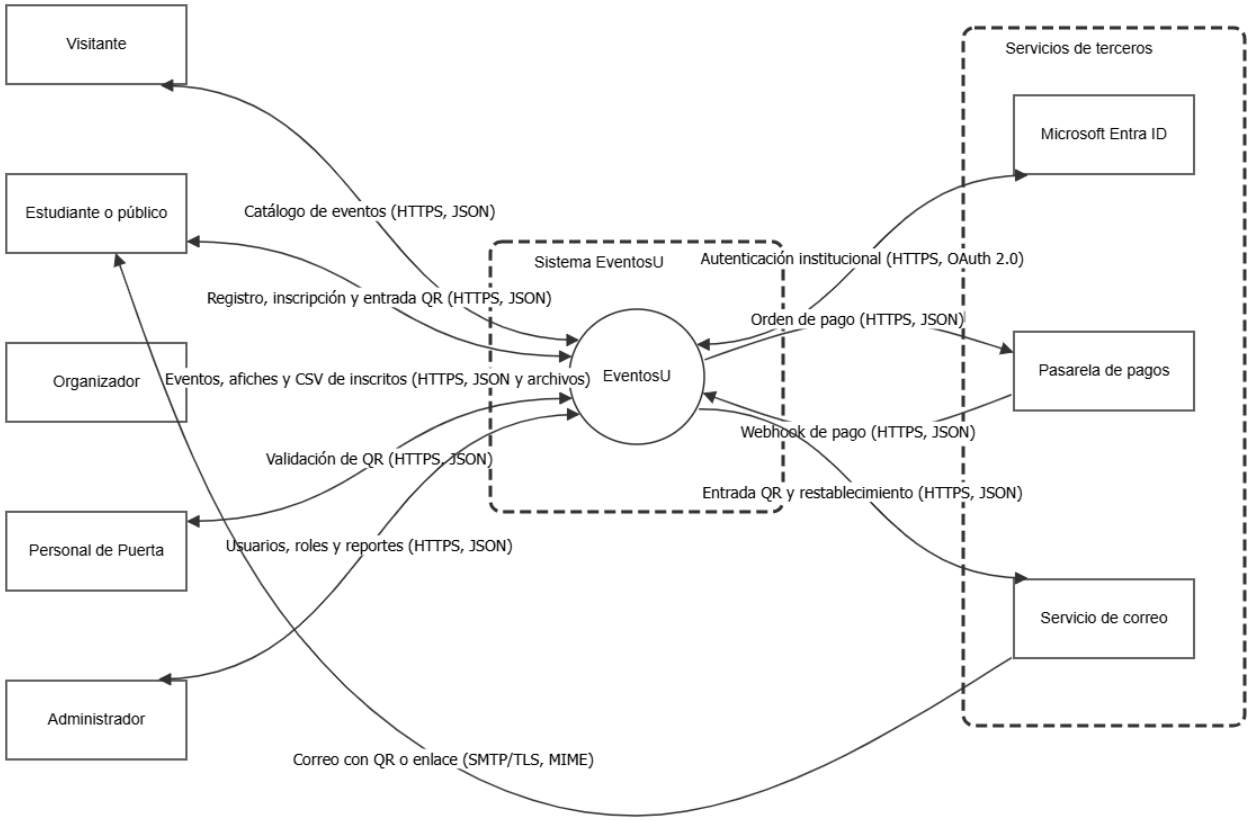
ID	Supuesto
S2	Todo el tráfico entre componentes usa HTTPS (TLS 1.2 o superior). La conexión entre la API y PostgreSQL usa TLS dentro de la red privada de la nube.
S3	"Guarda el token de sesión en el navegador" significa que el JWT está en `localStorage`, legible por cualquier JavaScript que corra en el sitio.
S4	El público externo se autentica con correo y contraseña, cuyo hash se guarda en PostgreSQL. Estudiantes, organizadores, personal de puerta y administradores usan su cuenta institucional con Entra ID.
S5	El rol del usuario viaja como un claim dentro del JWT que firma la API.
S6	El pago se hace en una página alojada por la pasarela (redirección). EventosU nunca ve el número de tarjeta; solo recibe el resultado por webhook.
S7	El QR contiene únicamente el identificador de la inscripción. La App de Puerta lo envía a la API y esta consulta la base de datos.
S8	La App de Puerta necesita conexión a internet para validar; no hay modo sin conexión.
S9	Hay un solo bucket para afiches y CSV. Los afiches se sirven con lectura pública para mostrarlos en el catálogo.
S10	La API genera el CSV, lo guarda en el bucket y entrega al organizador un enlace de descarga.
S11	La API y PostgreSQL están en la misma red privada; la base de datos no tiene dirección pública.
S12	El diseño descrito no menciona verificación de firma del webhook, límites de intentos ni registro de auditoría de acciones. Se asume que **no existen** hasta que un requisito los agregue.

3. DFD nivel 0 y nivel 1

El modelo está en `model/eventosu.json` y contiene dos diagramas. Las figuras de esta sección son capturas de OWASP Threat Dragon.

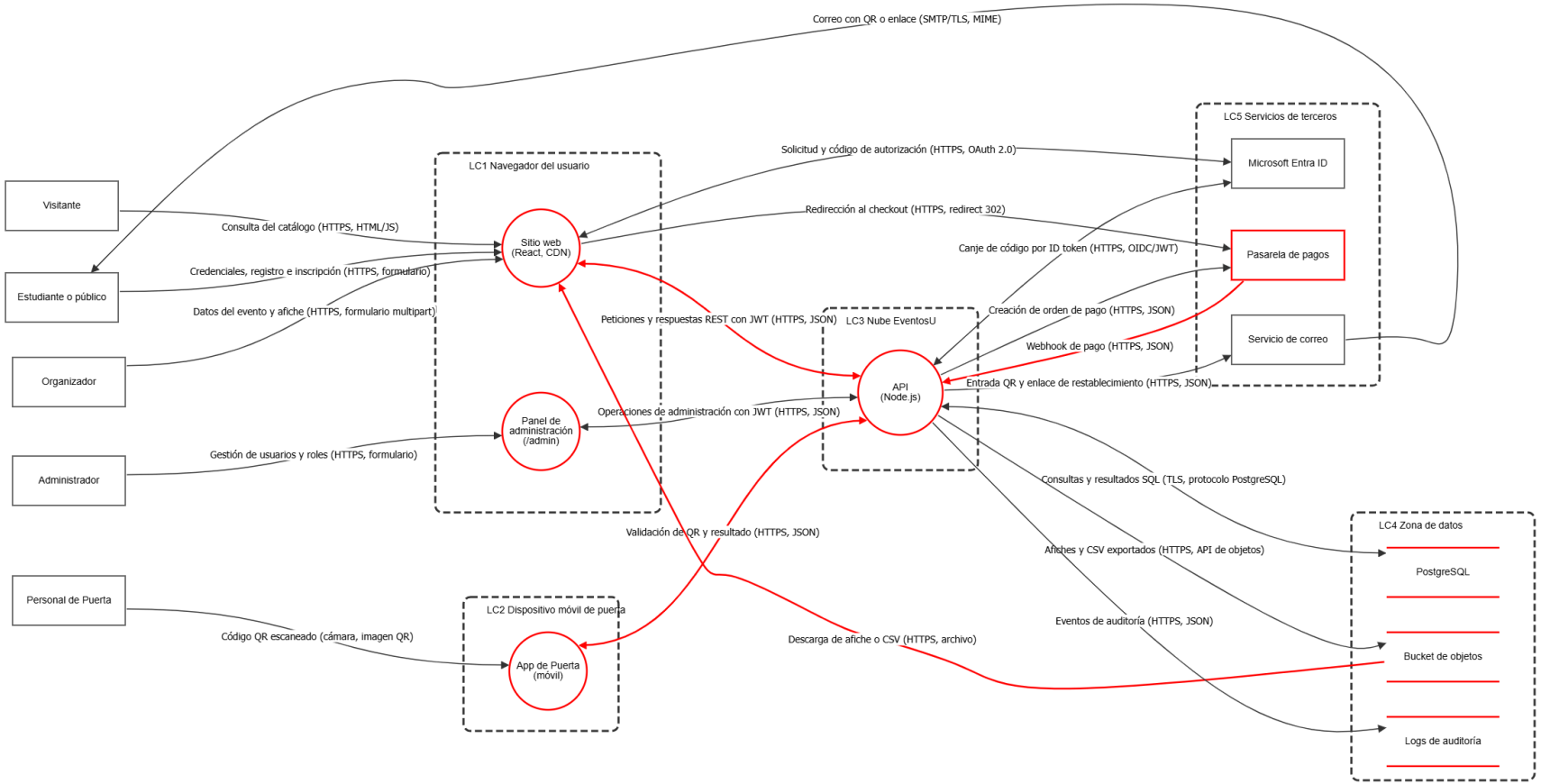
3.1 Nivel 0: contexto

EventosU como un solo proceso, sus cinco tipos de usuario y los tres servicios de terceros.



3.2 Nivel 1: componentes

Todos los componentes del caso con sus cinco límites de confianza (LC1 a LC5). Los elementos en rojo tienen amenazas abiertas. El detalle de cada flujo está en la sección 3.3.



3.3 Flujos de datos del nivel 1

ID	Origen → destino	Nombre del flujo (dato y protocolo)	Límite que cruza
F01	Visitante → Sitio web	Consulta del catálogo (HTTPS, HTML/JS)	Usuario → LC1
F02	Estudiante o público → Sitio web	Credenciales, registro e inscripción (HTTPS, formulario)	Usuario → LC1
F03	Organizador → Sitio web	Datos del evento y afiche (HTTPS, formulario multipart)	Usuario → LC1
F04	Administrador → Panel `/admin`	Gestión de usuarios y roles (HTTPS, formulario)	Usuario → LC1
F05	Personal de Puerta → App de Puerta	Código QR escaneado (cámara, imagen QR)	Usuario → LC2
F06	Sitio web ↔ API	Peticiones y respuestas REST con JWT (HTTPS, JSON)	LC1 ↔ LC3
F07	Panel `/admin` ↔ API	Operaciones de administración con JWT (HTTPS, JSON)	LC1 ↔ LC3
F08	App de Puerta ↔ API	Validación de QR y resultado (HTTPS, JSON)	LC2 ↔ LC3
F09	Sitio web ↔ Entra ID	Solicitud y código de autorización (HTTPS, OAuth 2.0)	LC1 ↔ LC5
F10	API ↔ Entra ID	Canje de código por ID token (HTTPS, OIDC/JWT)	LC3 ↔ LC5
F11	API → Pasarela	Creación de orden de pago (HTTPS, JSON)	LC3 → LC5
F12	Sitio web → Pasarela	Redirección al checkout (HTTPS, redirect 302)	LC1 → LC5
F13	Pasarela → API	Webhook de pago (HTTPS, JSON)	LC5 → LC3
F14	API → Servicio de correo	Entrada QR y enlace de restablecimiento (HTTPS, JSON)	LC3 → LC5
F15	Servicio de correo → Estudiante o público	Correo con QR o enlace (SMTP/TLS, MIME)	LC5 → usuario
F16	API ↔ PostgreSQL	Consultas y resultados SQL (TLS, protocolo PostgreSQL)	LC3 ↔ LC4
F17	API → Bucket	Afiches y CSV exportados (HTTPS, API de objetos)	LC3 → LC4
F18	Bucket → Sitio web	Descarga de afiche o CSV (HTTPS, archivo)	LC4 → LC1
F19	API → Logs de auditoría	Eventos de auditoría (HTTPS, JSON)	LC3 → LC4

3.4 Límites de confianza

ID	Límite	Por qué cambia la confianza
LC1	Navegador del usuario	El código React y el JWT viven en un equipo que controla el usuario. Todo lo que llega desde aquí puede estar manipulado.
LC2	Dispositivo móvil de puerta	El teléfono puede perderse, prestarse o analizarse; lo que la app guarde se considera extraíble.
LC3	Nube EventosU	Único lugar donde corre código que la universidad controla. Aquí se toman todas las decisiones de autorización.
LC4	Zona de datos	Solo la API debería hablar con estos almacenes. Su configuración (bucket público, permisos de logs) es responsabilidad de EventosU.
LC5	Servicios de terceros	EventosU no controla estos sistemas. Debe verificar que un mensaje realmente viene de ellos antes de actuar.

4. Tabla de amenazas

23 amenazas. Distribución: Spoofing 5, Tampering 6, Repudiation 3, Information disclosure 4, Denial of service 2, Elevation of privilege 3. Once involucran un flujo que cruza un límite de confianza (columna "Flujo que cruza límite").

Flujos obligatorios: **webhook de pago** (T01, T07), **validación del QR** (T05, T11, T12, T18, T20, T23) y **exportación del CSV** (T10, T15, T16).

La identificación (ID, elemento, STRIDE, escenario y flujo) está en esta tabla. La priorización (impacto, probabilidad, riesgo y justificación), la respuesta (estrategia y mitigación) y el estado de cada amenaza están en la sección 6.4. La tabla completa, con todas las columnas, vive en `docs/amenazas.csv`.

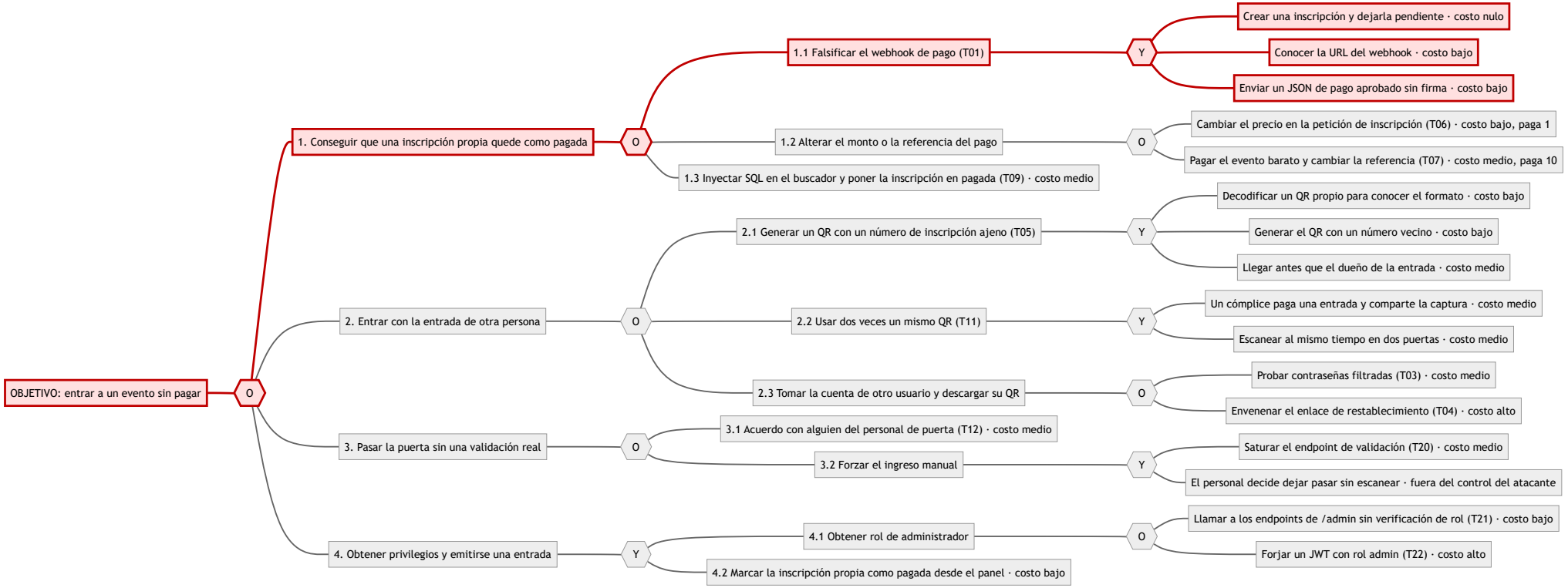
ID	Elemento	STRIDE	Escenario	Flujo que cruza límite
T01	Pasarela de pagos (emisor del webhook)	Spoofing	Un estudiante que crea una inscripción y deja el pago pendiente encuentra la URL del webhook (la ruta es predecible y aparece en la documentación pública de la pasarela). Envía con `curl` un JSON "pago aprobado" con su número de inscripción. La API no comprueba que el mensaje venga de la pasarela, marca la inscripción como pagada y le envía el QR por correo: obtiene una entrada sin pagar.	F13
T02	API (inicio de sesión institucional)	Spoofing	Un atacante envía a un organizador un enlace de "iniciar sesión con tu cuenta institucional" en el que cambió el `redirect_uri` por un dominio suyo. Si el registro de la aplicación en Entra ID acepta comodines, o la API no valida `state` ni usa PKCE, el atacante recibe el código de autorización, lo canjea y obtiene una sesión de organizador con acceso a las listas de inscritos.	F09
T03	API (inicio de sesión con correo y contraseña)	Spoofing	Un atacante prueba contra el endpoint de login miles de pares correo y contraseña filtrados de otros sitios. El diseño no limita intentos ni exige segundo factor al público externo, así que entra a las cuentas que reutilizan contraseña y descarga sus entradas QR.	No
T04	API (restablecimiento de contraseña)	Spoofing	Un atacante pide restablecer la contraseña de una víctima y altera la cabecera `Host` de la petición. La API arma el enlace con ese valor, el servicio de correo lo entrega a la víctima y, cuando ella hace clic, el token de restablecimiento llega al servidor del atacante, que toma la cuenta.	F15
T05	API (validación de QR)	Spoofing	Un asistente con una entrada legítima decodifica su QR y ve que solo contiene el número de inscripción `1042`. Genera con cualquier app un QR con `1041`, llega temprano y la API responde "válido": entra con la entrada de otra persona, que al llegar recibe "ya usado".	F08
T06	Flujo Sitio web ↔ API (inscripción)	Tampering	Un usuario intercepta con un proxy la petición de inscripción que sale de su navegador y cambia el campo de precio de 500 a 1. La API confía en el monto que envía el cliente al crear la orden en la pasarela; el usuario paga 1 y recibe una entrada de 500.	F06
T07	Flujo Pasarela → API (webhook de pago)	Tampering	Un usuario crea dos inscripciones: una a un taller de 10 y otra a un concierto de 500. Paga la barata y reenvía la notificación cambiando la referencia por la de la inscripción cara. La API solo revisa que el estado sea "pagado" y no compara monto ni orden contra lo que ella misma creó, así que marca como pagado el concierto.	F13

ID	Elemento	STRIDE	Escenario	Flujo que cruza límite
T08	Bucket de objetos (afiches)	Tampering	Un organizador, o alguien con su cuenta, sube un afiche y controla el nombre del objeto en la petición. Usa la ruta del afiche de otro evento y lo sobrescribe con una imagen que anuncia un cambio de sede falso y un QR de "pago" que lleva a una cuenta suya. Los visitantes ven el afiche falso con la marca de la universidad.	No
T09	PostgreSQL	Tampering	Un visitante sin cuenta escribe SQL en el buscador del catálogo. La API concatena el texto en la consulta, y el atacante ejecuta un `UPDATE` que pone en "pagada" su inscripción o cambia el cupo de un evento.	No
T10	Flujo Sitio web ↔ API (registro, dato que termina en el CSV)	Tampering	Una persona se registra con el nombre `=HYPERLINK("http://sitio-falso/login";"Ver detalle")`. La API guarda el texto tal cual y lo escribe en el CSV. Cuando el organizador abre el archivo en Excel, la celda se interpreta como fórmula y lo lleva a una página que le pide sus credenciales institucionales.	F06
T11	API (validación de QR)	Tampering	Dos amigos comparten la captura de pantalla de un solo QR y lo presentan al mismo tiempo en dos puertas. La API consulta el estado y después lo actualiza en dos pasos separados; ambas peticiones leen "no usado" y las dos puertas reciben "válido". Entran dos personas con una entrada.	No
T12	App de Puerta	Repudiation	Todo el personal de puerta comparte un mismo usuario genérico en la app. Un miembro del personal escanea por adelantado las entradas de sus conocidos o deja pasar gente sin escanear. Cuando un asistente reclama que su entrada aparece "ya usada", no se puede saber quién la validó, en qué puerta ni con qué teléfono, y el responsable lo niega.	No
T13	Panel de administración	Repudiation	Un administrador le asigna rol de organizador a un cómplice, este exporta la lista de inscritos y el administrador le quita el rol. Como los cambios de rol no dejan registro, el administrador niega haberlo hecho y no hay evidencia para contradecirlo.	No
T14	Logs de auditoría	Repudiation	Un atacante que ya comprometió la API usa la misma credencial con la que el contenedor escribe logs, que también permite borrar, y elimina las entradas de sus exportaciones de CSV. La universidad no puede determinar qué datos salieron ni notificar a los afectados.	No
T15	Flujo Bucket → Sitio web (descarga del CSV)	Information disclosure	El CSV de inscritos se guarda en el mismo bucket de lectura pública que los afiches, con un nombre predecible como `export-evento-87.csv`. Cualquier persona en internet prueba números de evento en la URL y descarga nombre, correo, carné y teléfono de todos los inscritos, sin iniciar sesión.	F18
T16	API (exportación de inscritos)	Information disclosure	El organizador del evento 87 cambia el identificador en la URL de exportación por 88. La API comprueba que tiene rol de organizador pero no que el evento sea suyo, y le entrega los datos personales de los inscritos de un evento ajeno.	No
T17	Sitio web	Information disclosure	Un organizador malicioso escribe un `<script>` en la descripción de su evento. El sitio la muestra sin escapar y el script se ejecuta en el navegador de cada persona que abre el evento, lee el JWT de `localStorage` y lo envía al atacante. Como `/admin` es el mismo sitio, si un administrador abre el evento el atacante obtiene un token de administrador.	No
T18	Flujo App de Puerta ↔ API (resultado de la validación)	Information disclosure	La respuesta de validación devuelve el registro completo de la inscripción (nombre, correo, carné, teléfono) aunque la app solo necesita "válido" o "ya usado". Alguien del personal de puerta, o quien encuentre un teléfono de puerta desbloqueado, envía números de inscripción consecutivos y recolecta los datos personales de todos los asistentes.	F08

ID	Elemento	STRIDE	Escenario	Flujo que cruza límite
T19	API (inscripción)	Denial of service	Un revendedor ejecuta un script el minuto en que abre la inscripción de un concierto y crea miles de inscripciones sin pagarlas. Cada una reserva un cupo sin fecha de vencimiento, el evento aparece agotado y los estudiantes reales no pueden inscribirse.	No
T20	Flujo App de Puerta ↔ API (validación de QR)	Denial of service	Media hora antes de un concierto, un atacante inunda de peticiones el endpoint de validación, que no limita la tasa, o la red wifi del recinto se satura. Como la app no tiene modo sin conexión, el personal no puede validar ninguna entrada: se forma una cola de cientos de personas o se decide dejar pasar sin escanear.	F08
T21	Panel de administración	Elevation of privilege	Un estudiante con sesión normal escribe `/admin` en la barra de direcciones. El sitio oculta el menú de administración solo en el navegador y los endpoints de administración únicamente verifican que el JWT sea válido, no el rol. El estudiante llama al endpoint de roles y se asigna el rol de administrador.	No
T22	API (verificación del JWT)	Elevation of privilege	Un usuario registrado decodifica su JWT, cambia el claim de rol a `admin` y lo firma de nuevo: adivina una clave HS256 débil con un diccionario, o envía el token con `alg: none` y la API lo acepta. Obtiene control total de usuarios, eventos y exportaciones.	No
T23	App de Puerta	Elevation of privilege	Un atacante descarga la App de Puerta, la descompila y encuentra una llave de API fija, compartida por todos los dispositivos. Con ella llama directamente a la API desde su computadora y marca como usada o disponible cualquier entrada, sin ser personal de puerta.	F08

5. Árbol de ataque: "entrar a un evento sin pagar"

Los nodos **O** se cumplen con cualquiera de sus hijos; los nodos **Y** necesitan todos. Cada hoja indica la amenaza relacionada y el costo estimado para el atacante. El camino más barato está resaltado en rojo.



Camino más barato

Objetivo → 1 → 1.1: falsificar el webhook de pago (T01).

Criterio	Camino 1.1 (webhook falso)	Alternativa más cercana
Dinero	Cero. No se paga nada.	1.2 exige pagar al menos 1 o 10; 2.2 exige que alguien pague una entrada completa.
Conocimiento	Saber enviar una petición HTTP con `curl` o Postman.	1.3 y 4.1 (T22) requieren técnica de inyección o criptografía.
Acceso previo	Solo una cuenta normal, que cualquiera puede crear.	3.1 necesita un cómplice dentro del personal.
Presencia y tiempo	Se hace desde casa, días antes del evento.	2.1 y 2.2 dependen de llegar antes que otro o de sincronizarse en dos puertas.
Rastro	La entrada resultante es indistinguible de una pagada; en la puerta no hay nada sospechoso.	2.1 deja a una víctima que reclama en la puerta.
Repetible	Sí: sirve para cualquier evento y cualquier cantidad de entradas, y se puede revender.	2.2 da una sola entrada extra por cada entrada pagada.

Las tres hojas de 1.1 son de costo nulo o bajo y ninguna depende de otra persona. El único dato que el atacante debe averiguar es la URL del webhook, y las pasarelas publican el formato de sus notificaciones. Por eso la verificación de la firma del webhook es el control con mejor relación costo-beneficio de todo el modelo: cierra el camino más barato con un cambio pequeño en la API.

El segundo camino más barato es 4.1 con T21 (endpoints de `/admin` sin verificación de rol), que también es gratuito y remoto, pero requiere descubrir los endpoints de administración y deja un cambio de rol visible para otros administradores.

6. Matriz de riesgo

La priorización usa una matriz de **impacto × probabilidad** de 3×3. Primero se califica cada amenaza en impacto y probabilidad con los criterios de abajo, aterrizados a EventosU; el cruce de ambos da el **riesgo** (Bajo, Medio, Alto o Crítico).

6.1 Criterios de impacto

Nivel	Cuándo aplica en EventosU
Alto	Pérdida económica directa y repetible (entradas gratis o con precio manipulado a escala), fuga masiva de datos personales (nombre, correo, carné, teléfono de muchos inscritos) o indisponibilidad total de la validación el día del evento. Daño reputacional serio para la universidad.
Medio	Fraude acotado (una o pocas entradas), fuga limitada de datos (un usuario o un evento) o degradación parcial del servicio. Recuperable con costo moderado.
Bajo	Afecta un registro o una entrada, sin fuga de datos personales a gran escala y recuperable sin costo relevante.

6.2 Criterios de probabilidad

Nivel	Cuándo aplica en EventosU
Alta	El atacante solo necesita herramientas básicas (`curl`, un proxy, el navegador) y ningún privilegio previo; el control ausente es evidente y el ataque es repetible sin barreras.
Media	Requiere un paso adicional (adivinar una URL, tener una cuenta, conocer la documentación, llegar antes que otro) o cierto contexto técnico u operativo.
Baja	Requiere técnica avanzada, un privilegio o cómplice interno, o depende de factores fuera del control del atacante (por ejemplo, que una víctima concreta haga clic).

6.3 Matriz 3×3 y escala de riesgo

Impacto \ Probabilidad	Baja	Media	Alta
Alto	Medio	Alto	**Crítico**
Medio	Bajo	Medio	Alto
Bajo	Bajo	Bajo	Medio

- **Crítico:** impacto alto + probabilidad alta. Es lo primero que hay que resolver antes de escribir código.
- **Alto:** impacto alto con probabilidad media, o impacto medio con probabilidad alta.
- **Medio / Bajo:** se resuelven después o se aceptan según el costo de mitigar.

6.4 Priorización y respuesta de las 23 amenazas

La identificación (escenario y flujo) está en la sección 4. Esta tabla añade impacto, probabilidad, riesgo, la respuesta elegida y el estado; la justificación resume en una línea el porqué de cada calificación. Como EventosU está en diseño, ninguna mitigación está implementada todavía: el estado es **Abierta** (mitigación definida, pendiente de construir) o **Aceptada**.

ID	Impacto	Probabilidad	Riesgo	Justificación	Respuesta	Mitigación	Estado
T01	Alto	Alta	Crítico	Entradas gratis a escala y reventa; solo `curl` y URL predecible, sin firma.	Mitigar	Verificar la firma HMAC-SHA256 del webhook con el secreto de la pasarela (gestor de secretos); descartar mensajes sin firma o con timestamp mayor a 5 min e ignorar notificaciones duplicadas.	Abierta
T02	Alto	Media	Alto	Sesión de organizador y listas de inscritos; requiere phishing a un organizador y configuración débil.	Mitigar	Lista blanca exacta de `redirect_uri` en Entra ID (sin comodines), validar `state` y usar PKCE con S256.	Abierta
T03	Medio	Alta	Alto	Cuentas individuales; el relleno de credenciales es masivo y automatizable.	Mitigar	Rate limiting por IP y cuenta en el login (5 intentos y bloqueo progresivo), MFA para roles elevados y verificación contra listas de contraseñas filtradas.	Abierta
T04	Medio	Media	Medio	Una cuenta a la vez; requiere envenenar `Host` y que la víctima haga clic.	Mitigar	Construir el enlace de restablecimiento con un dominio fijo configurado en el servidor, nunca desde la cabecera `Host`.	Abierta
T05	Medio	Alta	Alto	Una entrada por fraude; decodificar y regenerar un QR es trivial.	Mitigar	Firmar el QR con un token HMAC/JWT de expiración corta en lugar del ID secuencial; la API valida la firma.	Abierta
T06	Alto	Alta	Crítico	Pérdida de ingresos repetible; cambiar un campo con un proxy no requiere técnica.	Mitigar	Calcular el precio en el servidor (base de datos) y nunca usar el monto del cliente; validar monto y evento en el webhook contra la orden.	Abierta
T07	Alto	Media	Alto	Paga lo barato y recibe lo caro; requiere dos inscripciones y manipular la referencia.	Mitigar	Validar en el webhook que referencia, monto y evento coincidan con la orden creada por la API.	Abierta
T08	Medio	Media	Medio	Daño reputacional, no económico directo; requiere cuenta de organizador.	Mitigar	Nombre de objeto generado en el servidor (UUID), ACL de escritura solo para el organizador propietario, validar MIME/tamaño y versionado.	Abierta
T09	Alto	Media	Alto	Control total de la base de datos; requiere que exista concatenación y sea alcanzable.	Mitigar	Consultas parametrizadas u ORM en toda interacción con PostgreSQL; prohibir concatenación de entrada en SQL (SAST + revisión).	Abierta
T10	Medio	Media	Medio	Phishing dirigido al organizador; depende de abrir el CSV en Excel y hacer clic.	Mitigar	Al exportar CSV, prefijar con comilla simple (`'`) los campos que empiecen por `=`, `+`, `-`, `@` o tabulador.	Abierta
T11	Bajo	Media	Bajo	Una entrada extra por entrada pagada; requiere sincronizar dos puertas a la vez.	Mitigar	Marcar la entrada como usada con una operación atómica (`UPDATE ... WHERE estado = no_usado RETURNING`).	Abierta

ID	Impacto	Probabilidad	Riesgo	Justificación	Respuesta	Mitigación	Estado
T12	Medio	Baja	Bajo	Fraude limitado y sin rastro; requiere conducta indebida de personal propio.	Aceptar	Riesgo aceptado (ver 6.5). Control mínimo: registrar fecha, hora y dispositivo de cada validación.	Aceptada
T13	Medio	Baja	Bajo	Pérdida de rendición de cuentas; requiere un administrador malicioso y un cómplice.	Mitigar	Registrar en logs inmutables todo cambio de rol (quién, a quién, cuándo, valor anterior y nuevo).	Abierta
T14	Medio	Baja	Bajo	Agrava un incidente ya ocurrido; requiere haber comprometido la API antes.	Mitigar	Escritura de logs con identidad de solo-append separada de la lectura y almacén inmutable (object lock).	Abierta
T15	Alto	Alta	Crítico	Fuga masiva de datos personales; nombre predecible en bucket público, sin login.	Eliminar	Eliminar el CSV del bucket público: bucket privado y entrega por URL firmada de corta duración o endpoint autenticado.	Abierta
T16	Alto	Media	Alto	Expone PII de un evento ajeno; requiere cuenta de organizador.	Mitigar	Verificar propiedad del evento en la consulta (`WHERE evento_id = X AND organizador_id = sesion.uid`).	Abierta
T17	Alto	Media	Alto	Robo de token, incluido el de administrador; requiere inyectar script y que una víctima abra el evento.	Mitigar	Escapar la descripción al renderizar (sin `dangerouslySetInnerHTML`), CSP estricta y token en cookie HttpOnly + SameSite.	Abierta
T18	Medio	Media	Medio	Fuga de PII limitada; requiere acceso a un teléfono de puerta.	Mitigar	Devolver solo `{valido, motivo}` en la validación; nunca el registro completo.	Abierta
T19	Alto	Alta	Crítico	Evento agotado sin ventas; script trivial que reserva cupos sin pagar.	Mitigar	Expirar inscripciones no pagadas (15 min) con un job que libera el cupo y limitar pendientes por usuario e IP.	Abierta
T20	Alto	Media	Alto	Indisponibilidad en el peor momento; requiere inundar el endpoint o saturar la red.	Mitigar	Rate limiting en el endpoint de validación y modo sin conexión de la app con cola local que sincroniza.	Abierta
T21	Alto	Alta	Crítico	Control total del sistema; solo llamar endpoints que no validan rol.	Mitigar	Autorización por rol en el servidor (middleware) en todos los endpoints `/admin`; validar el claim de rol contra lista blanca.	Abierta
T22	Alto	Media	Alto	Control total; requiere forjar o adivinar la clave de firma.	Mitigar	Firmar el JWT con RS256 (clave en KMS), permitir solo RS256/ES256, rechazar `alg:none` y rotar claves.	Abierta
T23	Alto	Media	Alto	Manipular cualquier entrada; requiere descompilar la app y extraer la llave.	Mitigar	Autenticar cada dispositivo con credenciales propias y token corto; no embeber llaves fijas en la app.	Abierta

6.5 Riesgo aceptado: T12

Amenaza	T12: validaciones en la puerta sin responsable identificable (usuario compartido en la App de Puerta).
Riesgo	Bajo (impacto medio × probabilidad baja).
Decisión	Aceptar.
Justificación	El daño máximo son unas pocas entradas por evento y quien lo causa es personal asignado por la universidad, identificable por el turno y la puerta. Aunque el diseño prevé cuentas institucionales (S4), el personal de puerta suele ser temporal o voluntario; darle a cada uno una cuenta individual obliga a gestionar altas y bajas en cada evento, y ese costo operativo es mayor que la pérdida esperada.
Control mínimo	La API registra fecha, hora y dispositivo de cada validación, lo que permite acotar un reclamo a un teléfono y un turno.
Responsable	Coordinador de EventosU (dueño del servicio en la universidad), quien firma la aceptación.
Revisión	Se reevalúa si un evento acumula más de 5 reclamos de entrada "ya usada" o si se venden entradas de más de 500.

7. Trazabilidad de las 5 amenazas críticas

Las cinco amenazas de mayor riesgo (Crítico) se trazan a un requisito de OWASP ASVS 4.0.3, a una tarea con criterio de aceptación y al control que la verifica en el pipeline antes de producción.

Amenaza	Requisito ASVS	Historia de usuario / tarea (con criterio de aceptación)	Verificación en el pipeline
T01 — Falsificar el webhook de pago	**V13.2.6** — «Verify that the message headers and payload are trustworthy and not modified in transit… per-message digital signatures…» (el secreto se gestiona según **V2.10.4**).	**Como API**, al recibir un webhook solo marco una inscripción como pagada si la firma HMAC coincide y el timestamp es reciente. **Criterio:** un webhook sin firma o con firma inválida responde `401` y no cambia el estado; un webhook firmado válido actualiza solo la inscripción referenciada.	**Prueba manual / test de integración en CI** (SAST no cubre lógica de integración): enviar con `curl` un webhook sin firma → `401`; con firma válida → `200`.
T06 — Alterar el precio en la inscripción	**V11.1.5** — «Verify the application has business logic limits or validation to protect against likely business risks or threats, identified using threat modeling…».	**Como API**, calculo el precio desde mi base de datos y lo envío a la pasarela; ignoro cualquier monto que venga del cliente. **Criterio:** si un cliente altera el campo precio con un proxy, la orden enviada a la pasarela conserva el precio oficial del evento.	**Prueba manual / test unitario de negocio en CI**: modificar el precio con un proxy y verificar que la orden conserva el precio oficial (SAST no detecta lógica de negocio).
T15 — CSV de inscritos en bucket público	**V4.2.1** — «Verify that sensitive data and APIs are protected against Insecure Direct Object Reference (IDOR) attacks… viewing everyone's records…» (el almacenamiento se endurece con **V12.4.1**).	**Como usuario sin sesión**, no puedo descargar el CSV; solo el organizador propietario lo obtiene por URL firmada de corta duración. **Criterio:** al pedir `export-evento-87.csv` por URL pública el objeto no existe en el bucket público y la descarga exige sesión y propiedad del evento.	**IaC** (escaneo de la política del bucket con `checkov` / `tfsec`: sin permisos públicos) + **DAST** (intentar descargar el objeto sin sesión).
T19 — Acaparar cupos sin pagar	**V11.1.4** — «Verify that the application has anti-automation controls to protect against excessive calls such as mass data exfiltration, business logic requests, file uploads or denial of service» (límites por usuario en **V11.1.3**).	**Como sistema**, expiro inscripciones sin pagar a los 15 minutos, libero el cupo y limito inscripciones pendientes por usuario e IP. **Criterio:** un script que crea 1000 inscripciones sin pagar es limitado por usuario/IP y los cupos se liberan al vencer el plazo.	**Prueba manual / test de carga de negocio en CI** (simular una ráfaga de inscripciones sin pago) + **DAST** para verificar el rate limiting.
T21 — `/admin` sin verificación de rol	**V4.1.1** — «Verify that the application enforces access control rules on a trusted service layer, especially if client-side access control is present and could be bypassed» (menor privilegio en **V4.1.3**).	**Como API**, verifico el rol en el servidor en todos los endpoints `/admin`, no solo que el JWT sea válido. **Criterio:** un JWT válido con rol estudiante recibe `403` en cualquier endpoint `/admin` y no modifica roles ni datos.	**DAST** (prueba de control de acceso: token de estudiante contra endpoints `/admin` → `403`) + **SAST** (verificar que todos los endpoints `/admin` usan el middleware de autorización).

8. Conclusiones

El análisis muestra que el riesgo de EventosU se concentra en cuatro decisiones de diseño, todas corregibles **antes de escribir una línea de código**:

1. **Separar `/admin` del sitio público y verificar el rol en el servidor.** El panel no puede compartir el mismo login ni confiar en "ocultar el menú" en el navegador (T21); la autorización por rol debe vivir en la API (V4.1.1) y los administradores deben usar MFA (V4.3.1).
2. **Firmar el QR y validarlo de forma atómica.** El QR no puede ser un ID secuencial (T05): debe ser un token firmado (JWT/HMAC) de corta vida, y el marcado de "usado" debe ser una sola operación atómica para evitar el doble uso (T11).
3. **Verificar la firma del webhook y calcular el precio solo en el servidor.** Cierran de un golpe las tres formas más baratas de entrar gratis: webhook falso (T01), precio manipulado (T06) y referencia cambiada (T07). La firma HMAC del webhook es el control con mejor relación costo-beneficio de todo el modelo.
4. **Sacar los CSV del bucket público y expirar las inscripciones impagas.** Los datos personales nunca deben quedar en un objeto de lectura pública con nombre predecible (T15); los cupos no pagados deben liberarse solos para que un script no "agote" el evento (T19).

Con esos cuatro cambios se eliminan las cinco amenazas críticas y la mayoría de las de riesgo alto. El resto son controles puntuales (firma del JWT con RS256, consultas parametrizadas, validación de MIME/tamaño en afiches, rate limiting) que se aplican como historias técnicas en el primer sprint.